UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD INGENIERIA
ESCUELA DE CIENCIAS Y SISTEMAS
ANALISIS Y DISEÑO DE SISTEMAS 1
SECCION B
AUX. DOUGLAS MARTINEZ

Manual técnico

GRUPÖ 1	
Henry Alexander García Montúfar	201497949
Adriana Lucia Ojeda Rivas	202000363
Luis Fernando Gomez Rendon	201801391
Francisco Javier Cetino Mendez	202006716
Jorge Rolando Ambrosio Perez	201700408

Guatemala, 23 de febrero, de 2026

Sistema Web de Gestión de Notas

Sistema web fullstack que permite a los usuarios:

- Registrarse
- Iniciar sesión
- Crear notas
- Editar notas
- Eliminar notas
- Fijar (Pin)
- Archivar
- Compartir notas

Arquitectura:

Frontend (React)
Backend (Node + Express)
Base de Datos (PostgreSQL - Neon)

BACKEND

Tecnologías utilizadas

- Node.js
- Express.js
- JWT (jsonwebtoken)
- bcryptjs
- cors
- dotenv
- PostgreSQL
- Neon (base de datos en la nube)

Estructura del Backend

```
backend/
│
├── controllers/
│   ├── auth.controller.js
│   └── notes.controller.js
│
├── routes/
│   ├── auth.routes.js
│   └── notes.routes.js
```

```
│
├── config/
│   └── db.js
│
├── app.js
└── .env
```

Rutas de Autenticación

Base URL:

http://localhost:4000/api/auth

Registrar usuario

POST /register

Body:

```
{
  "usuario": "user",
  "password": "123456"
}
```

Respuesta:

```
{
  "message": "Usuario registrado correctamente"
}
```

Login

POST /login

Body:

```
{
  "usuario": "user",
  "password": "123456"
}
```

Respuesta:

{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter, Routes, Route } from "react-router-dom"

import Login from '../Modulos/Login/login.jsx'
import Home from '../Modulos/Home/home.jsx'
import Registrarse from '../Modulos/Resitro/Resitrarse.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <Routes>
        <Route path='/' element={<Login />} />
        <Route path='/home' element={ <Home /> } ></Route>
        <Route path='/registro' element={<Registrarse />} />
      </Routes>
    </BrowserRouter>
  </StrictMode>,
)
```

login.jsx

```
import { useState } from "react"
import { Link, useNavigate } from "react-router-dom"

function Login(){

  const [usuario, setUsuario] = useState("");
  const [password, setPassword] = useState("");
  const navigate = useNavigate();

  const Loguearse = async () => {


  }

    return (
  <div className="w-screen h-screen bg-gradient-to-br from-green-300 via-blue-400 to-purpl

    <div className="w-full h-[8%] bg-cyan-700 shadow-lg flex items-center justify-between
      <p className="text-3xl px-9 font-bold text-white tracking-wide">
        Mis Notas
      </p>
    </div>
    <div className="w-full h-[92%] flex">
      <div className="w-1/2 h-full flex justify-center items-center">
        <div className="bg-white/30 backdrop-blur-lg shadow-2xl rounded-2xl py-12 px-10 fl

          <div className="text-center">
            <h2 className="text-3xl font-bold text-white">Login</h2>
            <p className="text-white/80 text-sm mt-2">Bienvenido de nuevo</p>
          </div>
```

Resitrarse.jsx

```
import { useState, useEffect } from "react"
import { Link, useAsyncError, useNavigate } from "react-router-dom"
import AlertaError from "../Alertas/AlertaError";

function Registrarse(){

    const [usuario, setUsuario] = useState("");
    const [password, setPassword] = useState("");
    const [confpassw, setConfpassw ] = useState("");

    const [ messerror, setMesserror ] = useState(false);
    const [ txtmess, setTxtmess ] = useState("");

    const validarRegistro = async () =>{

        if ( password === "" || confpassw === "" || usuario === "" ) {
            setTxtmess("Es necesario rellenar todos los campos")
            setMesserror(true)
        } else if(  password !== confpassw  ){
            setTxtmess("La contraseña no coincide")
            setMesserror(true)
        }

    }

    useEffect( () =>{
        if ( messerror ) {
            const timer = setTimeout(() => {
                setMesserror(false)
            }, 3000)

            return () => clearTimeout(timer)
        }
```

Rutas de Notas

Base URL:

http://localhost:4000/api/notes

Requieren token JWT en el header:

Authorization: Bearer TOKEN

Método	Endpoint	Descripción
POST	/	Crear nota

Método	Endpoint	Descripción
GET	/	Obtener notas
PUT	/:id	Actualizar nota
DELETE	/:id	Eliminar nota
PATCH	/:id/pin	Fijar/Desfijar nota
PATCH	/:id/archive	Archivar/Desarchivar
POST	/:id/share	Compartir nota

FRONTEND

Tecnologías utilizadas

- React
- Vite
- React Router DOM
- Tailwind CSS
- Fetch API

Estructura del Frontend

```
frontend/
│
├── src/
│   ├── Modulos/
│   │   ├── Login/
│   │   ├── Home/
│   │   └── Registro/
│   │
│   ├── main.jsx
│   └── App.jsx
```

Flujo de Autenticación

1. Usuario ingresa credenciales.
2. React envía POST a /api/auth/login.

3. Backend valida y devuelve token.
4. Token se guarda en:

```
localStorage.setItem("token", data.token);
```

5. Para consumir notas:

```
fetch("http://localhost:4000/api/notes", {
  headers: {
    Authorization: `Bearer ${token}`
  }
});
```

main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter, Routes, Route } from "react-router-dom"

import Login from '../Modulos/Login/login.jsx'
import Home from '../Modulos/Home/home.jsx'
import Registrarse from '../Modulos/Resitro/Resitrarse.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <Routes>
        <Route path='/' element={<Login />} />
        <Route path='/home' element={ <Home /> } ></Route>
        <Route path='/registro' element={<Registrarse />} />
      </Routes>
    </BrowserRouter>
  </StrictMode>,
)
```

login.jsx

```jsx
import { useState } from "react"
import { Link, useNavigate } from "react-router-dom"

function Login(){

  const [usuario, setUsuario] = useState("");
  const [password, setPassword] = useState("");
  const navigate = useNavigate();

  const Loguearse = async () => {


  }

    return (
  <div className="w-screen h-screen bg-gradient-to-br from-green-300 via-blue-400 to-purpl

    <div className="w-full h-[8%] bg-cyan-700 shadow-lg flex items-center justify-between
      <p className="text-3xl px-9 font-bold text-white tracking-wide">
        Mis Notas
      </p>
    </div>
    <div className="w-full h-[92%] flex">
      <div className="w-1/2 h-full flex justify-center items-center">
        <div className="bg-white/30 backdrop-blur-lg shadow-2xl rounded-2xl py-12 px-10 fl

          <div className="text-center">
            <h2 className="text-3xl font-bold text-white">Login</h2>
            <p className="text-white/80 text-sm mt-2">Bienvenido de nuevo</p>
          </div>

          <div className="flex flex-col gap-y-5">
```

resitrarse.jsx

```
end > Modulos > Resitro > Resitrarse.jsx > Registrarse
import { useState, useEffect } from "react"
import { Link, useAsyncError, useNavigate } from "react-router-dom"
import AlertaError from "../Alertas/AlertaError";

function Registrarse(){

    const [usuario, setUsuario] = useState("");
    const [password, setPassword] = useState("");
    const [confpassw, setConfpassw ] = useState("");

    const [ messerror, setMesserror ] = useState(false);
    const [ txtmess, setTxtmess ] = useState("");

    const validarRegistro = async () =>{

        if ( password === "" || confpassw === "" || usuario === "" ) {
            setTxtmess("Es necesario rellenar todos los campos")
            setMesserror(true)
        } else if(  password !== confpassw  ){
            setTxtmess("La contraseña no coincide")
            setMesserror(true)
        }

    }

    useEffect( () =>{
        if ( messerror ) {
            const timer = setTimeout(() => {
                setMesserror(false)
            }, 3000)

            return () => clearTimeout(timer)
        }
```

BASE DE DATOS

Motor utilizado

- PostgreSQL
- Hospedado en Neon

Página oficial:
Neon

Modelo de Base de Datos

Tabla users

```
CREATE TABLE users (
   id SERIAL PRIMARY KEY,
   usuario VARCHAR(50) UNIQUE NOT NULL,
   password TEXT NOT NULL,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabla notes

```
CREATE TABLE notes (
   id SERIAL PRIMARY KEY,
   user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
   title VARCHAR(100),
   content TEXT,
   is_pinned BOOLEAN DEFAULT FALSE,
   is_archived BOOLEAN DEFAULT FALSE,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE notes (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    user_id INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

db.js

```
backend > config > JS db.js > default
import pkg from 'pg';
const { Pool } = pkg;

const pool = new Pool({
    connectionString: process.env.DATABASE_URL,
    ssl: {
        rejectUnauthorized: false
    }
});

export default pool;
```

Seguridad

- Contraseñas encriptadas con bcrypt.
- Autenticación mediante JWT.
- Middleware para validar token.
- CORS configurado.

Cómo ejecutar el proyecto

Backend

cd backend
npm install
npm run dev

Servidor:

http://localhost:4000

Frontend

```
cd frontend
npm install
npm run dev
```

Servidor:

http://localhost:5173

Variables de Entorno

Archivo .env:

```
PORT=4000
DATABASE_URL=postgresql://...
JWT_SECRET=clave_super_secreta
```

Conclusión

El sistema implementa:

- Arquitectura MVC
- Autenticación con JWT
- CRUD completo
- Seguridad básica
- Base de datos relacional
- Conexión en la nube